



Europäische Union



IKR
Institut für angewandte
KI und Robotik



uniA UNIVERSITÄT
AUGSBURG
Centre for Future Production

PraxisLab KI & Robotik

Grundlagen der Robotik

Kollaborative Roboter

Anleitung für den praktischen Einsatz von Computer Vision mit einem
Elephant Robotics myCobot280 Roboter



Inhaltsverzeichnis

1	Grundlagen	2
1.1	<i>Elephant Robotics myCobot 280</i>	2
1.1.1	Drag Teaching	2
1.1.2	Roboter Set-Up	3
1.2	<i>myBlockly</i>	4
1.2.1	Verbindung zum Roboter	4
1.2.2	Funktionen	6
2	Programme erstellen	7
2.1	<i>Punkte anfahren</i>	7
2.1.1	Schritt 1 – Punkte auswählen	7
2.1.2	Schritt 2 – Netzwerk	8
2.1.3	Schritt 3 – Python Skript	9
2.2	<i>Pick & Place Abläufe</i>	11
2.2.1	Schritt 1 – Punkte auswählen	11
2.2.2	Schritt 2 – Python Skript	11
2.3	<i>Einbindung des Vision-Modells</i>	13
2.3.1	Schritt 1 – Punkte auswählen	13
2.3.2	Schritt 2 – Modell einfügen	13
2.3.3	Schritt 3 – Python Skript	13

1 Grundlagen

1.1 Elephant Robotics myCobot 280

1.1.1 Drag Teaching

Um den Roboter eine bestimmte Bewegung wiederholt verfahren zu lassen, können wir das sog. **drag teaching** verwenden. Hier bewegen wir den Roboter mit der Hand und der Roboter wiederholt die Bewegung.

1. Navigiere hierzu im Menü am Roboterfuß zu **Maincontrol**.



2. Halte ab jetzt den Roboter fest, da sich die Bremsen lösen. Wähle **Record** → **Ram** um aufzunehmen.



3. Bewege den Roboter beliebig. Drücke einen beliebigen Knopf, sobald Du fertig bist.



4. Drücke auf **Play** → **Ram** → **Play**. Der Roboter wiederholt die gespeicherte Bewegung nun in Endlosschleife. Drücke **Stop**, um die Bewegung zu beenden.



5. Drücke **Stop** → **Exit**, um das Programm zu beenden.

1.1.2 Roboter Set-Up

Um den Roboter tatsächlich programmieren zu können, müssen wir zuerst den richtigen Modus auswählen.

Navigiere hierzu im Robotertermenü zu **Transponder** → **USB UART**.



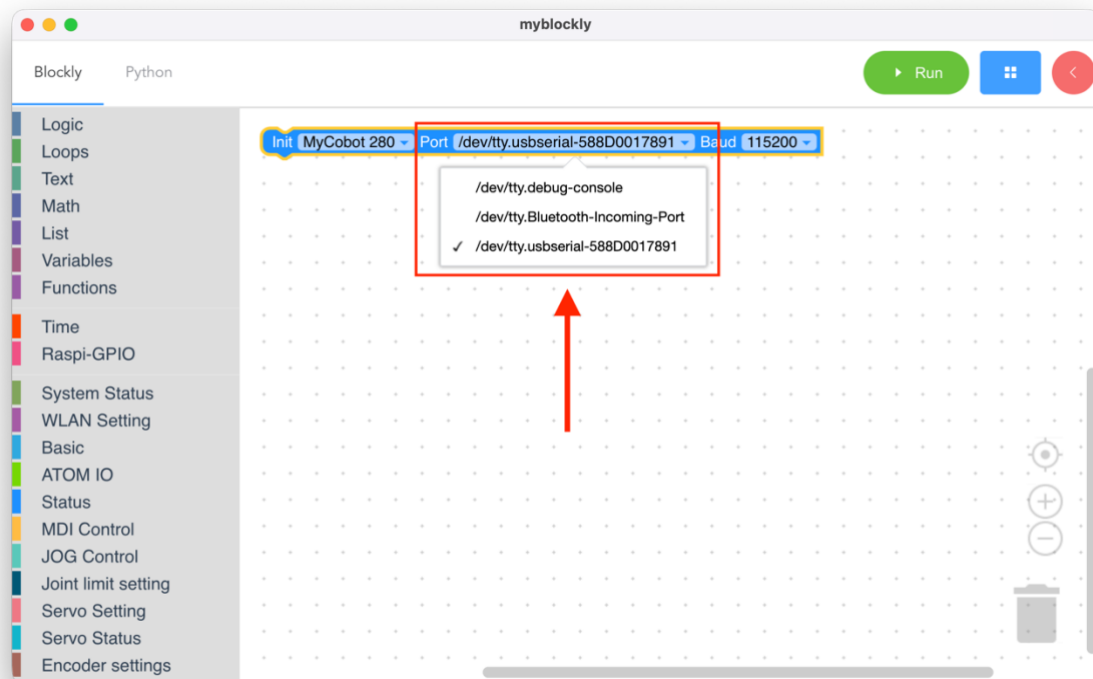
Sobald **Atom: ok** angezeigt wird, ist der Roboter bereit.

1.2 myBlockly

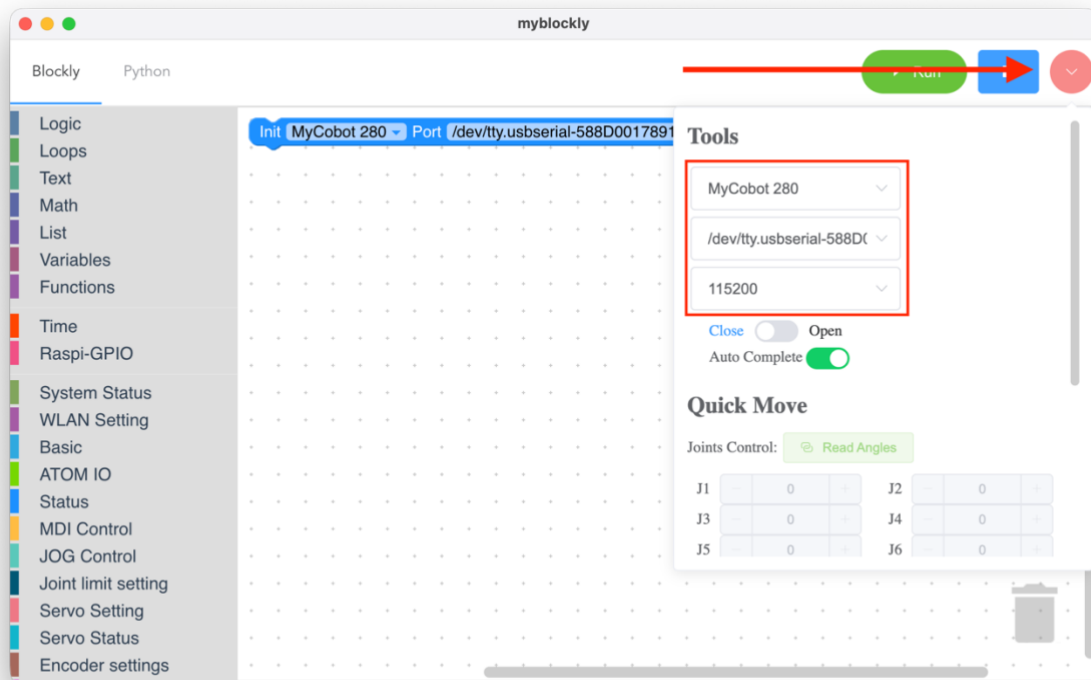
Um den Roboter anzusteuern, gibt es verschiedene Möglichkeiten. Neben dem drag teaching kann auch die Software **myBlockly** verwendet werden. Mit dieser kann der Roboter Direkt angesteuert und Programme graphisch erstellt werden.

1.2.1 Verbindung zum Roboter

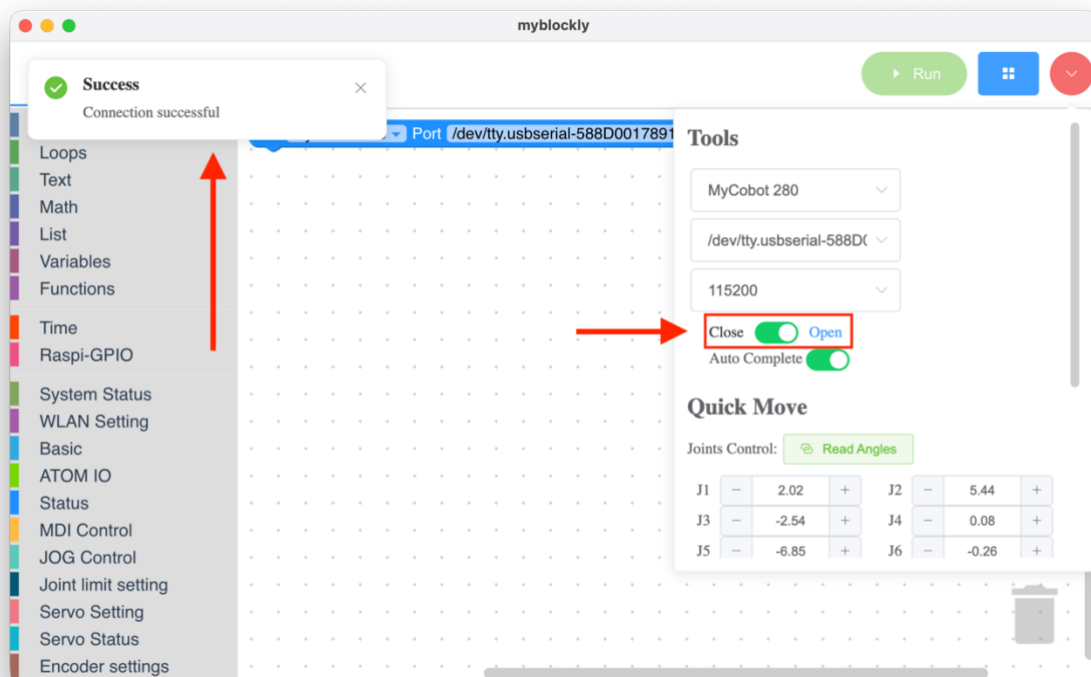
Um eine Verbindung zum Roboter aufzubauen, muss zunächst der richtige Port ausgewählt werden. Wähle hierfür aus der Drop-Down-Liste den entsprechenden Port aus.



Öffne jetzt auf der rechten Seite die Toolbar durch Klicken des roten Kreises. Achte darauf, dass hier der richtige Port übernommen wurde und die anderen Einstellungen übereinstimmen.



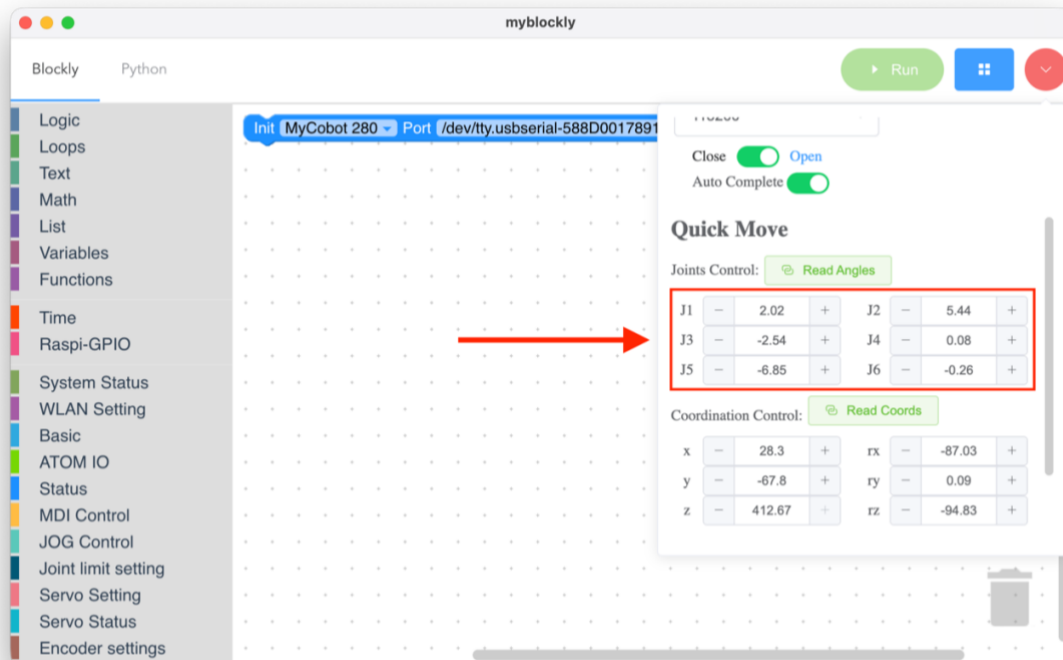
Öffne jetzt die Verbindung zum Roboter, durch Auswahl von **Open** in der Toolbar. Die erfolgreiche Verbindung erkennst Du an der Statusmeldung, die oben links auftaucht.



1.2.2 Funktionen

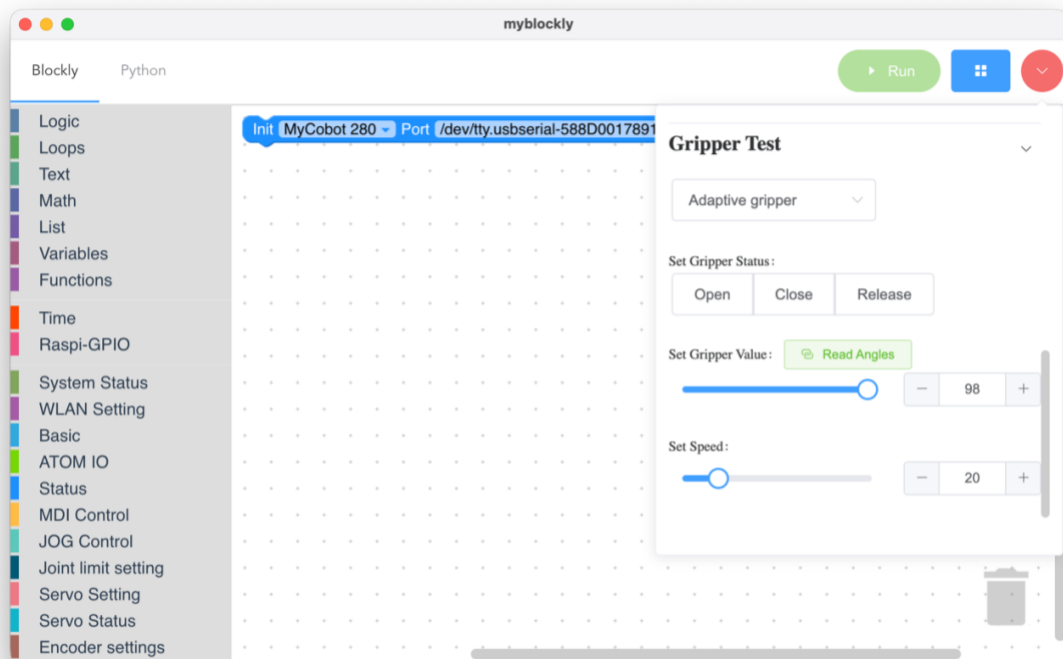
1.2.2.1 Quick Move

Über die Funktion Quick Move können die Achsen des Roboters einzeln angesteuert werden. Bewege alle Achsen des Roboters einzeln durch Klicken der + und – Buttons.



1.2.2.2 Gripper Test

Auch der Greifer kann so angesteuert werden. Versuche zudem damit ein Profil zu greifen und bewege es mit der Quick Move Funktion von der vorherigen Aufgabe.



2 Programme erstellen

Jetzt soll sich der Roboter auch gezielt bewegen können. Hierfür kannst Du drei vorbereitete Python Skripte verwenden. Öffne dazu den Ordner

S1_PraxisLab_KI_Robotik im Programm Visual Studio Code.

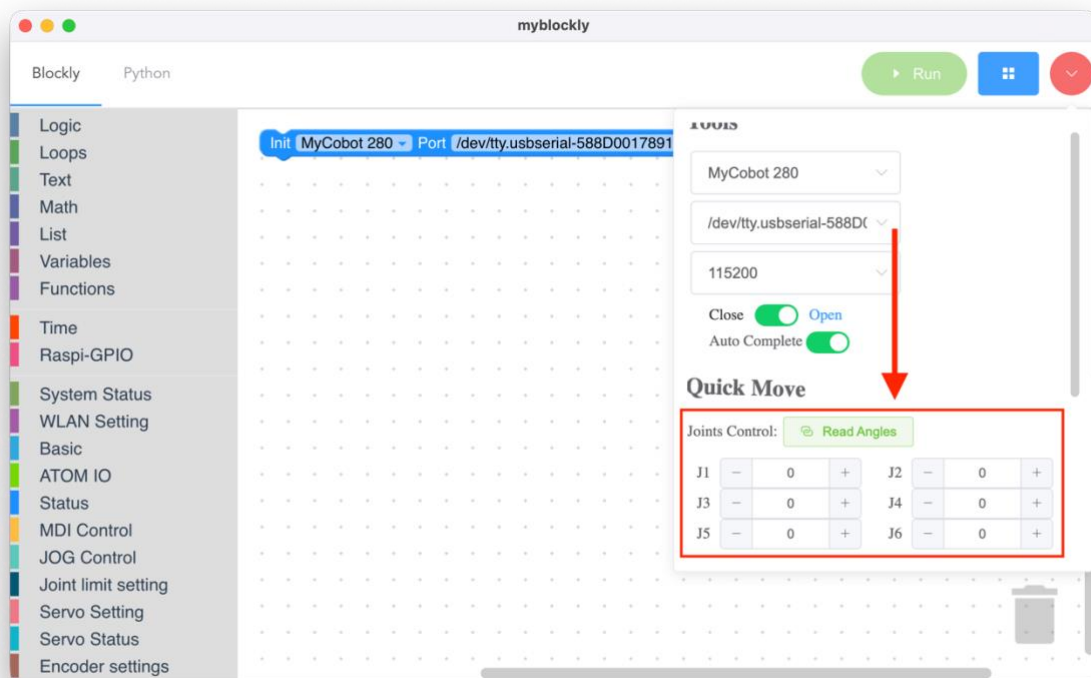
2.1 Punkte anfahren

Als ersten Schritt soll sich der Roboter zwischen zwei Punkten hin- und herbewegen.

2.1.1 Schritt 1 – Punkte auswählen

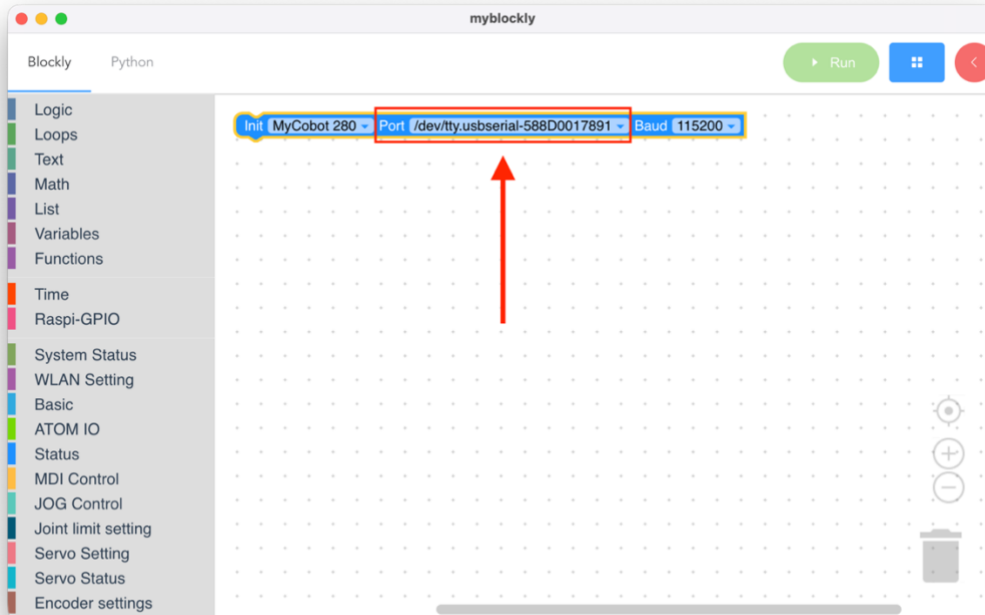
Verwende die Quick Move Funktion, um die Achswinkel von zwei beliebigen Punkten festzulegen. Merke Dir diese Punkte im Format [J1, J2, J3, J4, J5, J6].

Wichtig: Achte auf die richtige Reihenfolge der Winkel J1 bis J6!

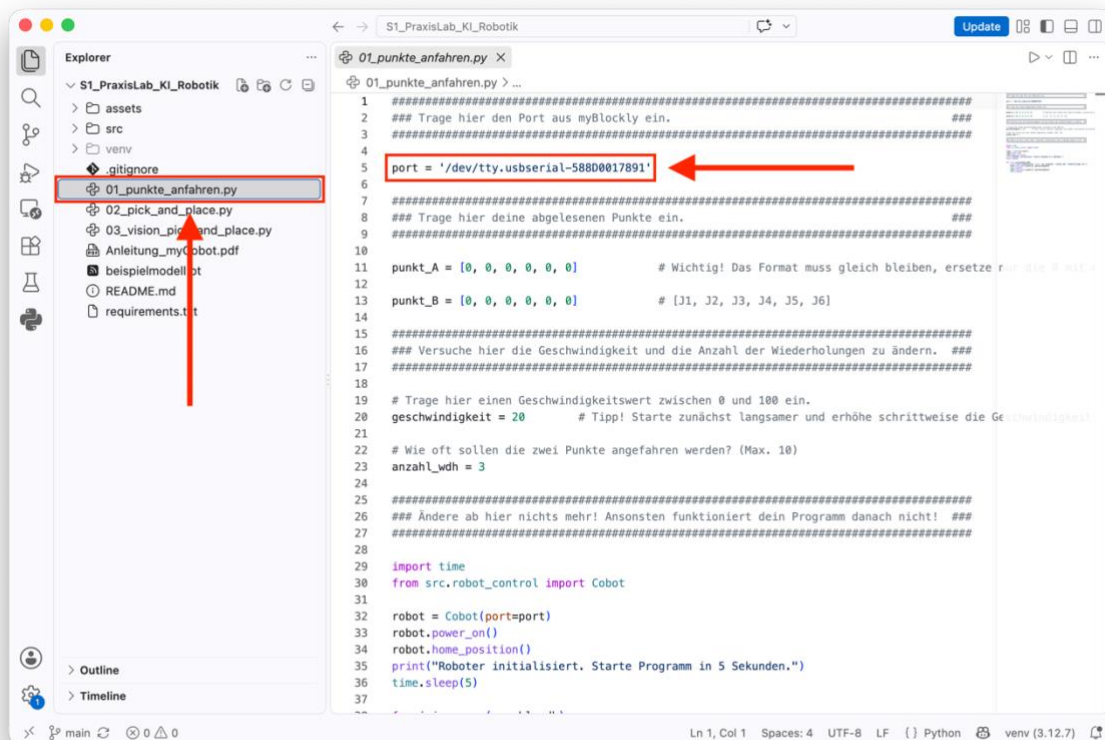


2.1.2 Schritt 2 – Netzwerk

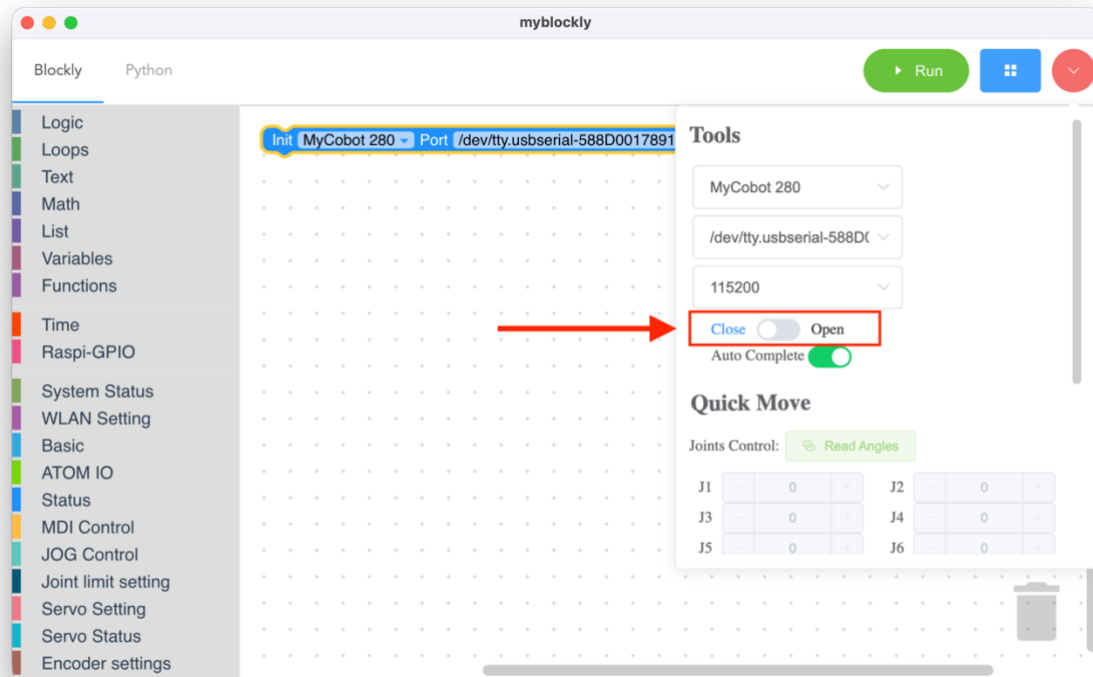
Übertrage den Port von myBlockly in das Python Skript und schließe anschließend die Verbindung in myBlockly.



Öffne die Datei **01_punkte_anfahren.py** in Visual Studio Code und trage den Port in der Zeile 5 ein.

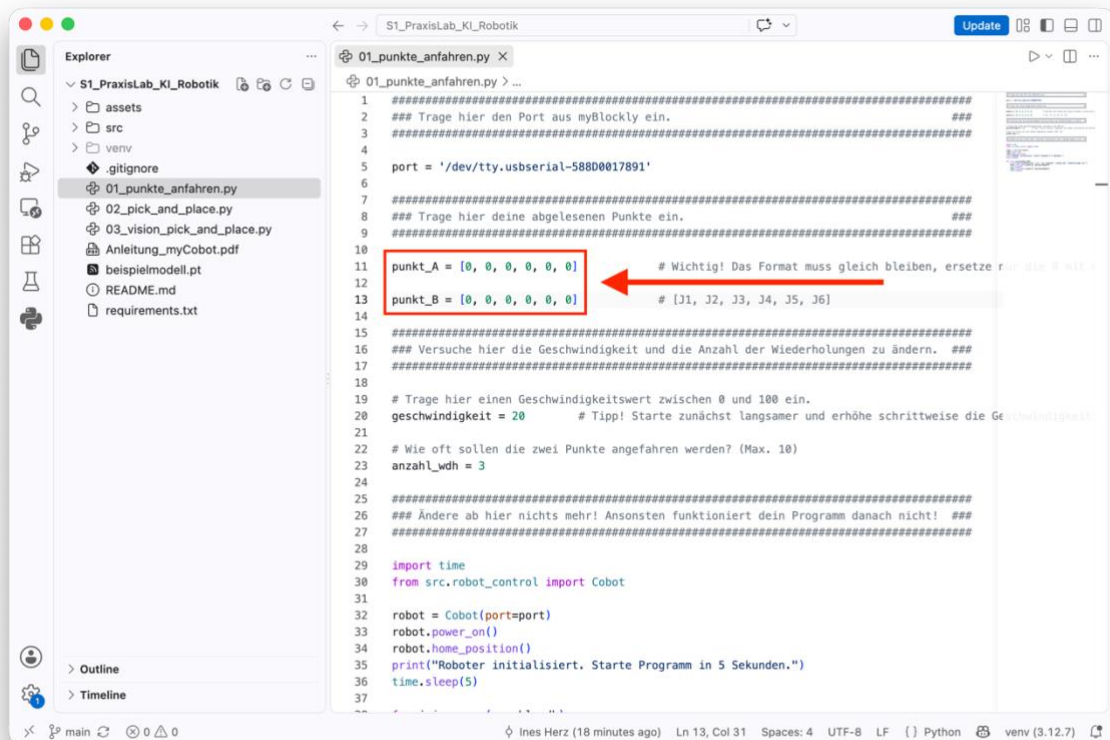


Schließe nun die Verbindung in myBlockly.



2.1.3 Schritt 3 – Python Skript

Trage Deine Punkte an den markierten Stellen im Python Skript im Format [J1, J2, J3, J4, J5, J6] ein.



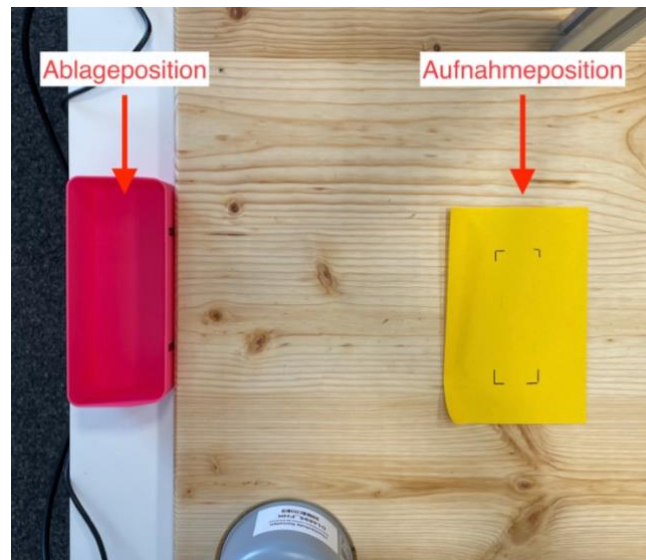
Führe das Skript aus.

```
1 #####
2  ## Trage hier den Port aus myBlockly ein.  ##
3  #####
4
5  port = '/dev/tty.usbserial-588D0017891'
6
7  #####
8  ## Trage hier deine abgelesenen Punkte ein.  ##
9  #####
10
11 punkt_A = [0, 0, 0, 0, 0] # Wichtig! Das Format muss gleich bleiben, ersetze nur die 0 mit x
12
13 punkt_B = [0, 0, 0, 0, 0] # [J1, J2, J3, J4, J5, J6]
14
15 #####
16 ## Versuche hier die Geschwindigkeit und die Anzahl der Wiederholungen zu ändern. ##
17 #####
18
19 # Trage hier einen Geschwindigkeitswert zwischen 0 und 100 ein.
20 geschwindigkeit = 20 # Tipp! Starte zunächst langsamer und erhöhe schrittweise die Geschwindigkeit!
21
22 # Wie oft sollen die zwei Punkte angefahren werden? (Max. 10)
23 anzahl_wdh = 3
24
25 #####
26 ## Ändere ab hier nichts mehr! Ansonsten funktioniert dein Programm danach nicht! ##
27 #####
28
29 import time
30 from src.robot_control import Cobot
31
32 robot = Cobot(port=port)
33 robot.power_on()
34 robot.home_position()
35 print("Roboter initialisiert. Starte Programm in 5 Sekunden.")
36 time.sleep(5)
37
```

Der Roboter sollte nun zwischen Deinen beiden Positionen hin- und herfahren. Du kannst versuchen die Geschwindigkeit des Roboters in Zeile 20 und die Anzahl der Wiederholungen in Zeile 3 verschieden einzustellen.

2.2 Pick & Place Abläufe

Als nächsten Schritt wollen wir eine Pick & Place Aufgabe umsetzen. Der Roboter soll in der Lage sein, ein 10cm langes Profil von einer festen Position aufzuheben und zu einer festen Zielposition zu bringen.



2.2.1 Schritt 1 – Punkte auswählen

Für diese Aufgabe müssen drei Punkte bestimmt werden. Bewege wieder den Roboter mithilfe der Quick Move Funktion in die Positionen und merke Dir die Achswinkel. Vergiss nicht die Verbindung in myBlockly vorher zu starten und am Ende wieder zu beenden!

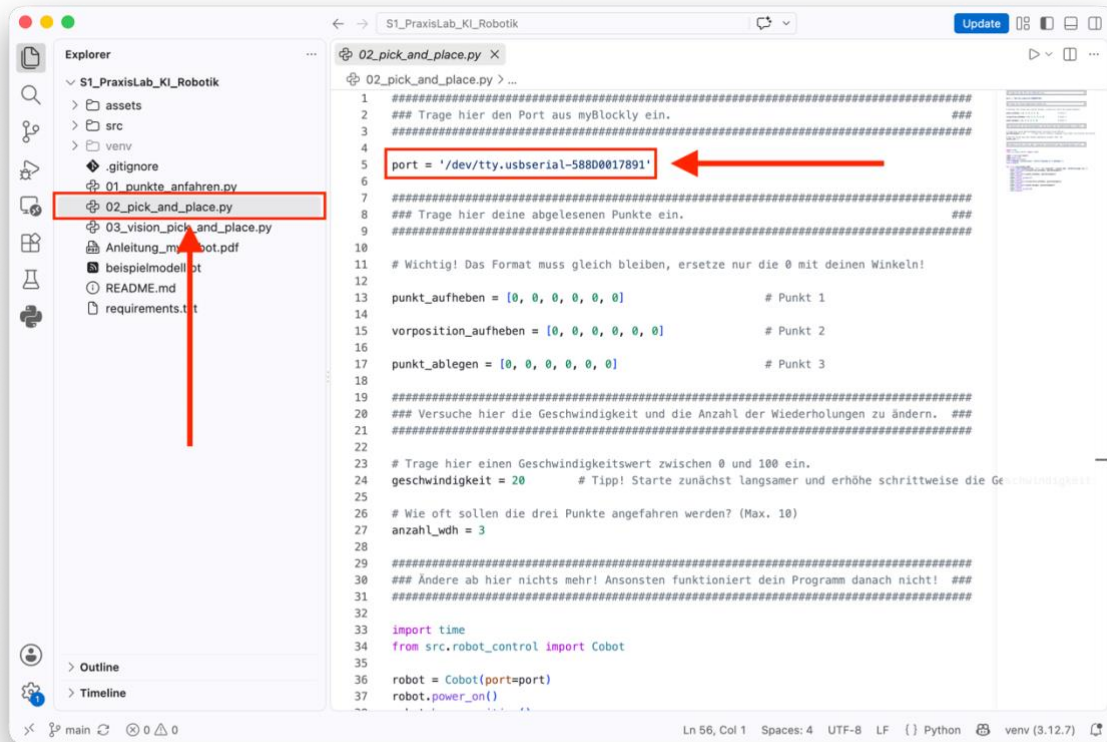
Am **ersten Punkt** hebt der Roboter das Bauteil auf. Wähle einen beliebigen Punkt und markiere ihn am besten auf einem Klebezettel.

Der **zweite Punkt** ist ca. 10 cm über dem Ersten. Diese Position fährt der Roboter an, bevor er das Teil aufhebt, damit er es nicht aus Versehen bewegt.

Der **dritte Punkt** ist die Ablageposition. Wähle diese am besten ca. 5 cm über der Kiste. Der Roboter lässt das Teil dann in die Kiste fallen.

2.2.2 Schritt 2 – Python Skript

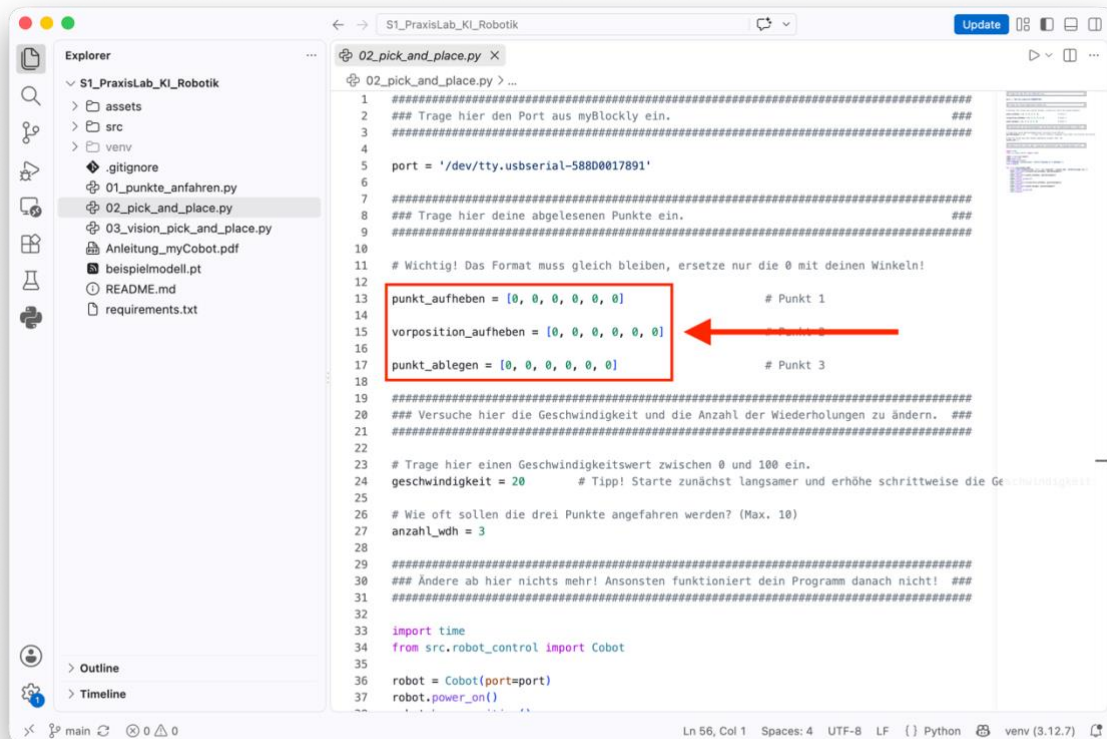
Öffne nun die Datei **02_pick_and_place.py** und trage auch hier den richtigen Port in Zeile 5 ein.



```

1 #####
2 ## Trage hier den Port aus myBlockly ein. #####
3 #####
4
5 port = '/dev/tty.usbserial-588D0017891'
6
7 #####
8 ## Trage hier deine abgelesenen Punkte ein. #####
9 #####
10
11 # Wichtig! Das Format muss gleich bleiben, ersetze nur die 0 mit deinen Winkeln!
12
13 punkt_aufheben = [0, 0, 0, 0, 0, 0] # Punkt 1
14
15 vorposition_aufheben = [0, 0, 0, 0, 0, 0] # Punkt 2
16
17 punkt_ablegen = [0, 0, 0, 0, 0, 0] # Punkt 3
18
19 #####
20 ## Versuche hier die Geschwindigkeit und die Anzahl der Wiederholungen zu ändern. ##
21 #####
22
23 # Trage hier einen Geschwindigkeitswert zwischen 0 und 100 ein.
24 geschwindigkeit = 20 # Tipp! Starte zunächst langsamer und erhöhe schrittweise die Geschwindigkeit!
25
26 # Wie oft sollen die drei Punkte angefahren werden? (Max. 10)
27 anzahl_wdh = 3
28
29 #####
30 ## Ändere ab hier nichts mehr! Ansonsten funktioniert dein Programm danach nicht! ##
31 #####
32
33 import time
34 from src.robot_control import Cobot
35
36 robot = Cobot(port=port)
37 robot.power_on()
  
```

Füge Deine abgelesenen Punkte in die Zeilen 13, 15, und 17 ein.



```

1 #####
2 ## Trage hier den Port aus myBlockly ein. #####
3 #####
4
5 port = '/dev/tty.usbserial-588D0017891'
6
7 #####
8 ## Trage hier deine abgelesenen Punkte ein. #####
9 #####
10
11 # Wichtig! Das Format muss gleich bleiben, ersetze nur die 0 mit deinen Winkeln!
12
13 punkt_aufheben = [0, 0, 0, 0, 0, 0] # Punkt 1
14
15 vorposition_aufheben = [0, 0, 0, 0, 0, 0] # Punkt 2
16
17 punkt_ablegen = [0, 0, 0, 0, 0, 0] # Punkt 3
18
19 #####
20 ## Versuche hier die Geschwindigkeit und die Anzahl der Wiederholungen zu ändern. ##
21 #####
22
23 # Trage hier einen Geschwindigkeitswert zwischen 0 und 100 ein.
24 geschwindigkeit = 20 # Tipp! Starte zunächst langsamer und erhöhe schrittweise die Geschwindigkeit!
25
26 # Wie oft sollen die drei Punkte angefahren werden? (Max. 10)
27 anzahl_wdh = 3
28
29 #####
30 ## Ändere ab hier nichts mehr! Ansonsten funktioniert dein Programm danach nicht! ##
31 #####
32
33 import time
34 from src.robot_control import Cobot
35
36 robot = Cobot(port=port)
37 robot.power_on()
  
```

Führe das Python Skript aus. Lege hierfür händisch jeweils ein Profil an die Aufnahme position. Wenn der Roboter es genommen hat, kann das Nächste an dieselbe Stelle gelegt werden.

2.3 Einbindung des Vision-Modells

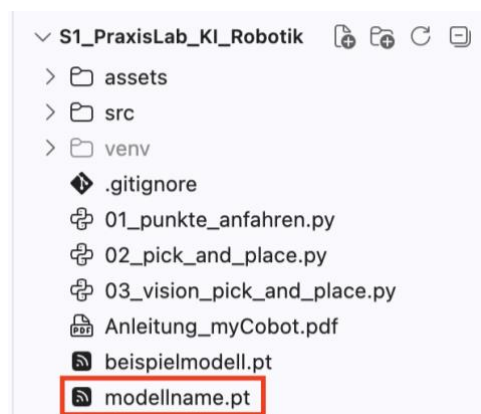
Als letzte Aufgabe wollen wir das zuvor trainierte Modell mit dem Roboter verbinden. Der Roboter soll in der Lage sein über das Kamerabild und das Vision-Modell die Position der Profile zu bestimmen, sie aufzuheben, und auf die Ablageposition zu bringen.

2.3.1 Schritt 1 – Punkte auswählen

Bestimme wie in der vorherigen Aufgabe die Ablageposition für die Profile über die Quick Move Funktion. Die Position sollte wieder etwa 5 cm über der Kiste liegen.

2.3.2 Schritt 2 – Modell einfügen

Kopiere als nächstes Dein trainiertes Modell in den Ordner. Deine Ordnerstruktur sollte folgendermaßen aussehen:



2.3.3 Schritt 3 – Python Skript

Öffne die Datei **03_vision_pick_and_place.py** in Visual Studio Code. Füge nun in Zeile 5 den Port, in Zeile 13 die Achswinkel für die Ablageposition, und in Zeile 21 den Namen Deines Modells ein.

Lege nun Profile möglichst senkrecht in den Arbeitsbereich des Roboters und führe das Skript aus. Der Roboter holt nun der Reihe nach die Profile und bewegt sie auf die Ablageposition.





Europäische Union

